Air University

Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 24, 2025

Assignment No. 02

Question 01

Design a combinational circuit that takes three inputs (A, B, C) and produces an output that is 1 only when exactly two of the inputs are 1.

- **Draw the truth table.**
- **Simplify the Boolean expression by any of these methods (e.g. Boolean Algebra Rules, De-Morgan's Theorem, K-Map)**
- **Implement the circuit on Proteus.**

+ Question 02

Design a digital circuit that generates an output of HIGH when an even number of inputs (A, B, C) are HIGH. Derive the Boolean expression for this requirement, simplify it using Boolean algebra, and implement the circuit.

Solution (question 1 + 2)

1. Truth Table:

A	B	C	$\overline{A}$	$\overline{B}$	$\overline{C}$	AB	$AB\overline{C}$	$A\overline{B}$	$A\overline{B}C$	$\overline{A}B$	$\overline{A}BC$	$AB\overline{C} + A\overline{B}C$	$AB\overline{C} + A\overline{B}C + \overline{A}BC$
0	0	0	1	1	1	0	**0**	0	**0**	0	**0**	**0**	**0**
0	0	1	1	1	0	0	**0**	0	**0**	0	**0**	**0**	**0**
0	1	0	1	0	1	0	**0**	0	**0**	1	**0**	**0**	**0**
0	1	1	1	0	0	0	**0**	0	**0**	1	**1**	**0**	**1**
1	0	0	0	1	1	0	**0**	1	**0**	0	**0**	**0**	**0**
1	0	1	0	1	0	0	**0**	1	**1**	0	**1**	**1**	**1**
1	1	0	0	0	1	1	**1**	0	**0**	0	**0**	**1**	**1**
1	1	1	0	0	0	1	**0**	0	**0**	0	**0**	**0**	**0**

2. Simplifying the Boolean Expression:

To derive the Boolean expression for the output being 1 when exactly two inputs are 1, we start with the following truth table:

Initial Expression:

Output = $(A \cdot B \cdot \overline{C}) + (A \cdot \overline{B} \cdot C) + (\overline{A} \cdot B \cdot C)$

Simplification Using Boolean Algebra:

Our objective is to express the condition where exactly two inputs are 1. Thus, we can streamline our initial expression by looking at all valid combinations:

Output = $(A \cdot B \cdot \overline{C}) + (A \cdot \overline{B} \cdot C) + (\overline{A} \cdot B \cdot C)$

This expression captures our requirement accurately. For clarity, it can also be rewritten as:

Output = $A\bar{B}C + AB\bar{C} + \bar{A}BC$

However, further simplification through Boolean rules won't alter the meaning of this expression.

3. Implementation in Proteus:

Components Needed:

- Logic Gates: AND, OR, NOT
- Inputs: A, B, C
- Output: Y = $A\bar{B}C + AB\bar{C} + \bar{A}BC$

Circuit Design:

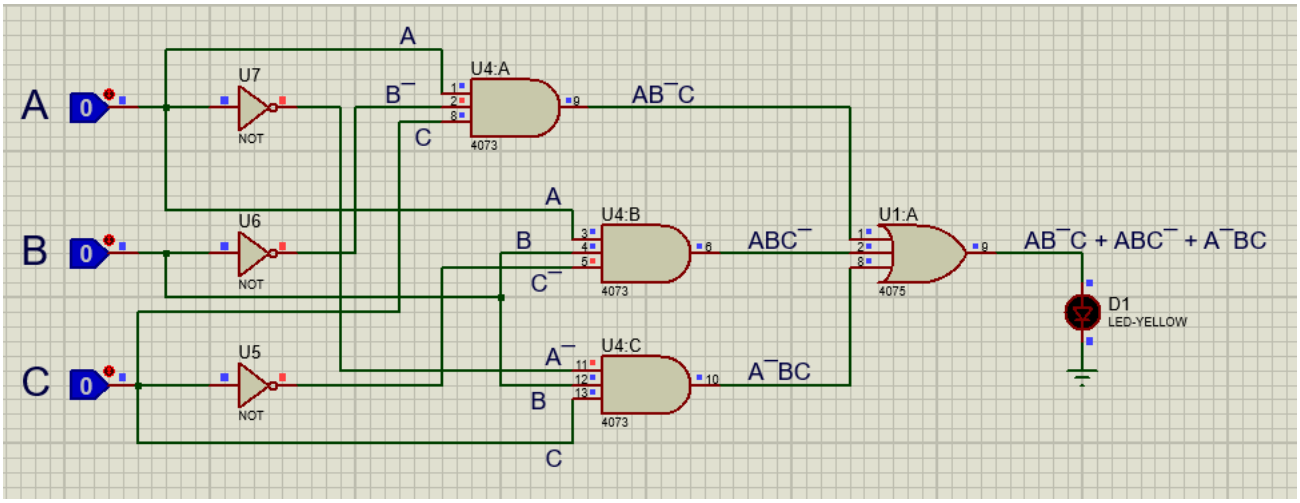
1. **AND Gates:** Use three AND gates to implement the expressions $A\bar{B}C$, $AB\bar{C}$, and $\bar{A}BC$.
2. **NOT Gates:** Employ NOT gates to obtain $\bar{B}$ and $\bar{C}$ from B and C, as well as $\bar{A}$ from A.
3. **OR Gate:** Connect the outputs from the three AND gates to an OR gate to produce the final output Y.

Implementation Steps:

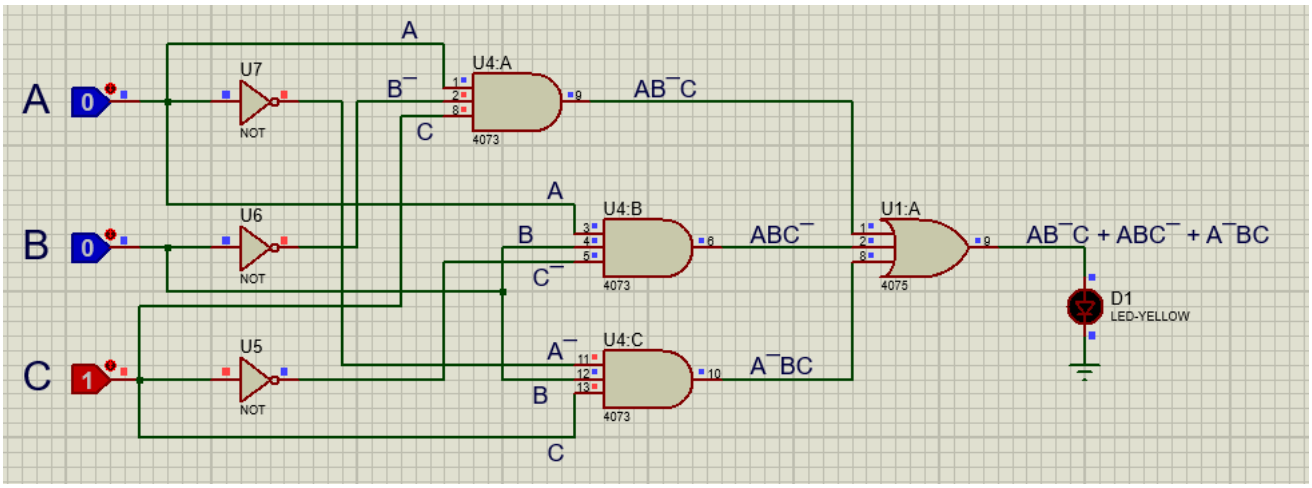
1. Place three AND gates (7408) in the Proteus environment.
2. Set up three NOT gates (7404) to invert A, B, and C.
3. Connect the inputs as follows:
   a. AND Gate 1: A, $\bar{B}$ (from NOT gate), C
   b. AND Gate 2: A, B, $\bar{C}$ (from NOT gate)
   c. AND Gate 3: $\bar{A}$ (from NOT gate), B, C
4. Link the outputs from the three AND gates to an OR gate (7432).
5. The output from the OR gate will serve as the final output Y.

Verification:

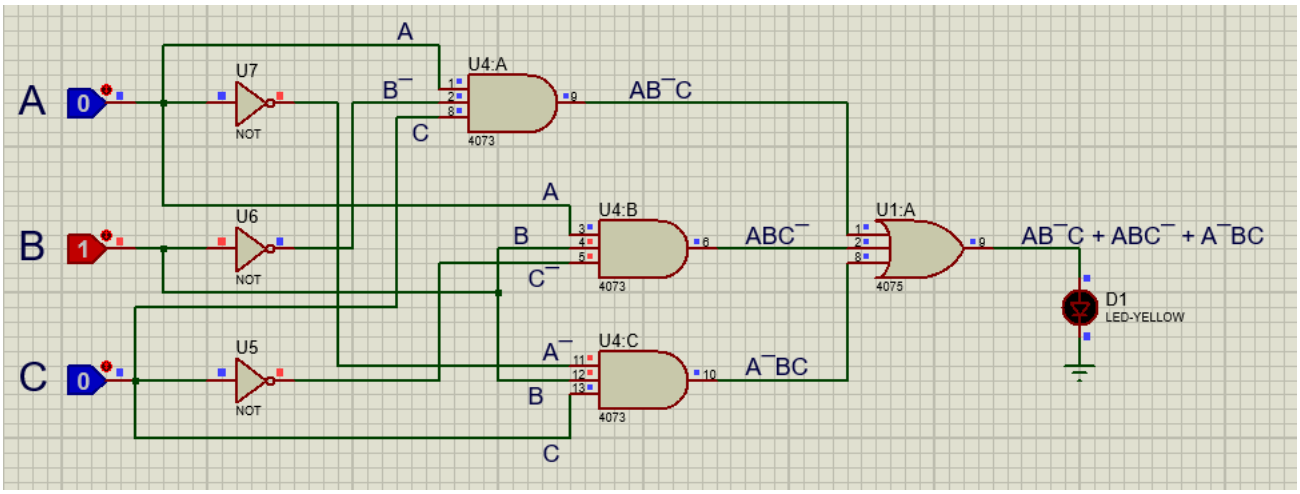
Input: A = 0, B = 0, C = 0 → **Output: 0**



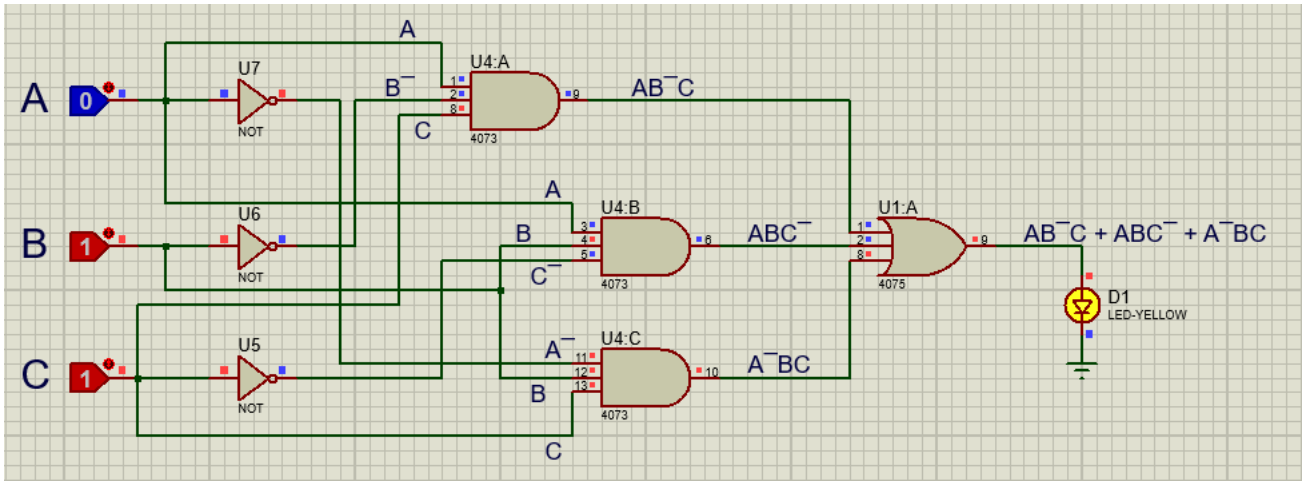
Input: A = 0, B = 0, C = 1 → **Output: 0**



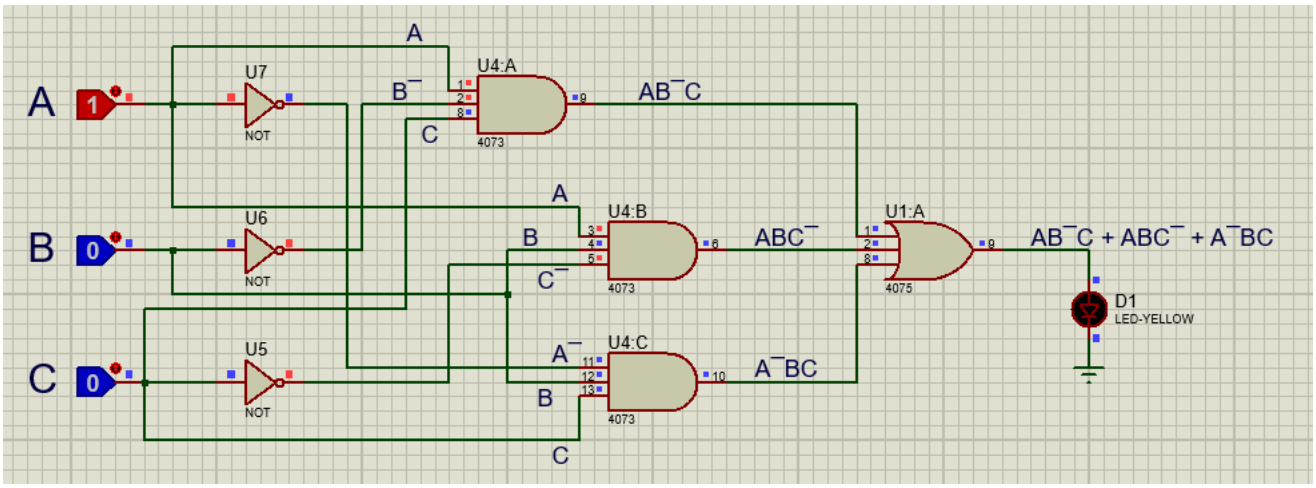
Input: A = 0, B = 1, C = 0 → **Output: 0**



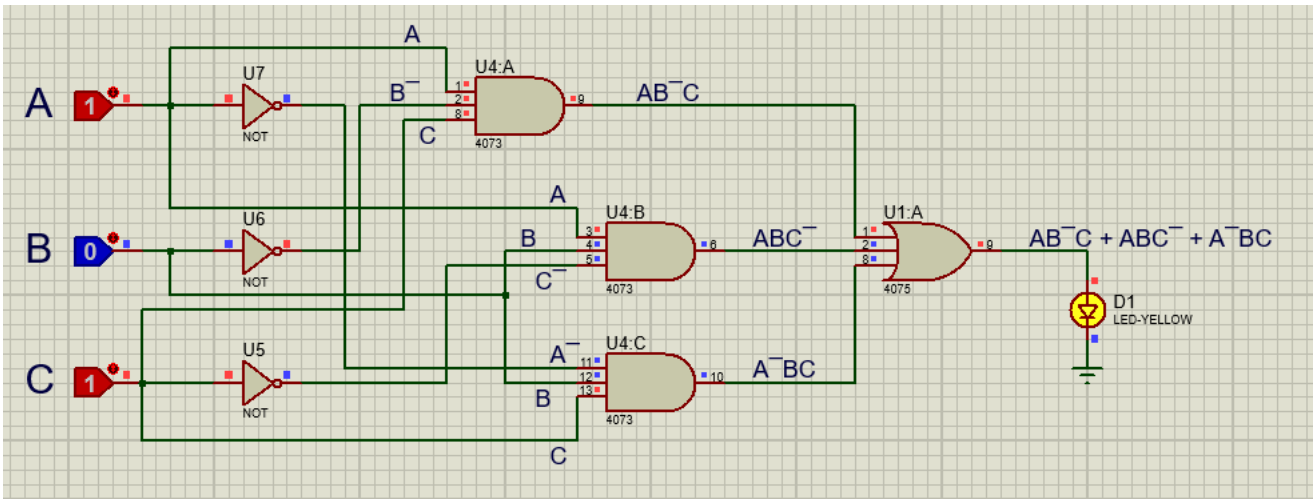
Input: A = 0, B = 1, C = 1 → **Output: 1**



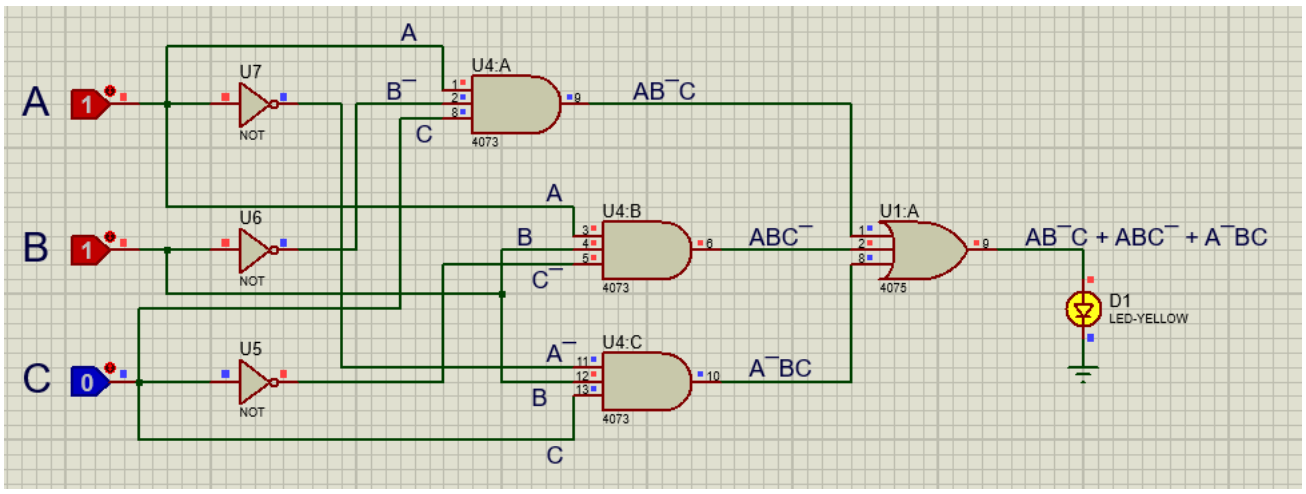
Input: A = 1, B = 0, C = 0 → **Output: 0**



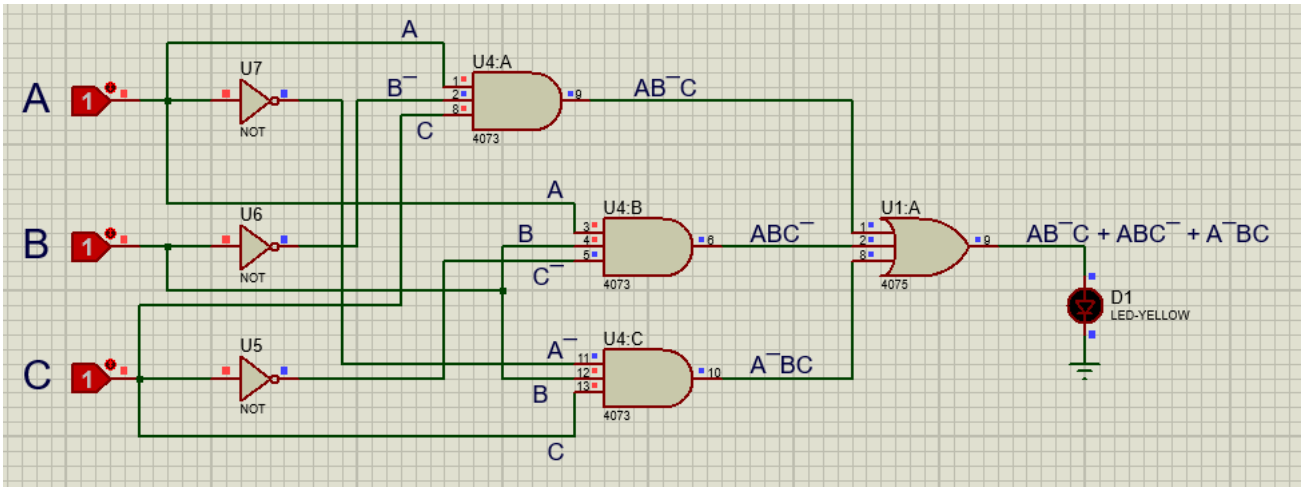
Input: A = 1, B = 0, C = 1 → **Output: 1**



Input: A = 1, B = 1, C = 0 → **Output: 1**



Input: A = 1, B = 1, C = 1 → **Output: 0**



Question 03

Design Full Adder Circuit

- **Draw Truth Table and Write Logic Expression**
- **Implement Circuit on Proteus**

Design Full Subtractor Circuit

- **Draw Truth Table and Write Logic Expression**
- **Implement Circuit on Proteus**

Solution

1. Full Adder Circuit

i. Truth Table and Logic Expression

A	B	C_in	Sum (S)	C_out
0	0	0	**0**	**0**
0	0	1	**1**	**0**
0	1	0	**1**	**0**
0	1	1	**0**	**1**
1	0	0	**1**	**0**
1	0	1	**0**	**1**
1	1	0	**0**	**1**
1	1	1	**1**	**1**

Logic Expressions:

- Sum (S) = A ⊕ B ⊕ C_in
- Carry (C_out) = (A · B) + (C_in · (A ⊕ B))

ii. Implementation on Proteus:

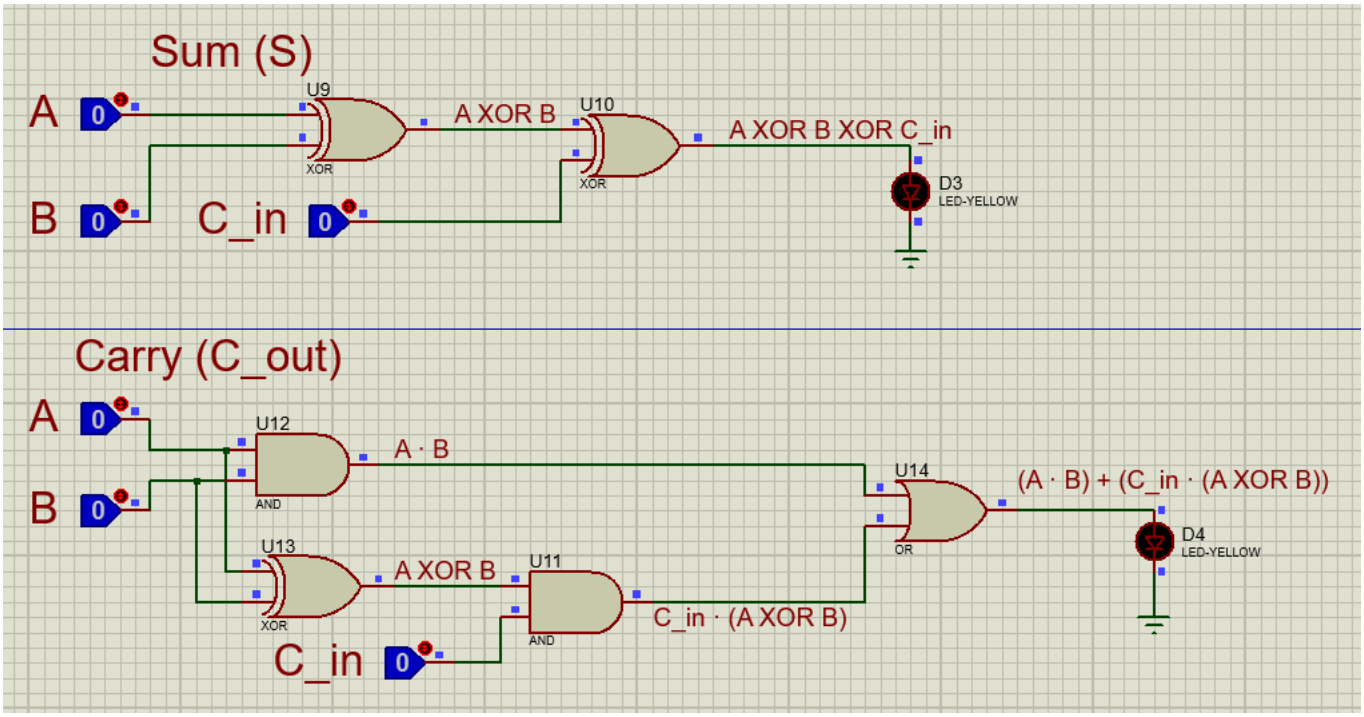
Components Needed: XOR Gates (7486), AND Gates (7408), OR Gate (7432).

Circuit Design:

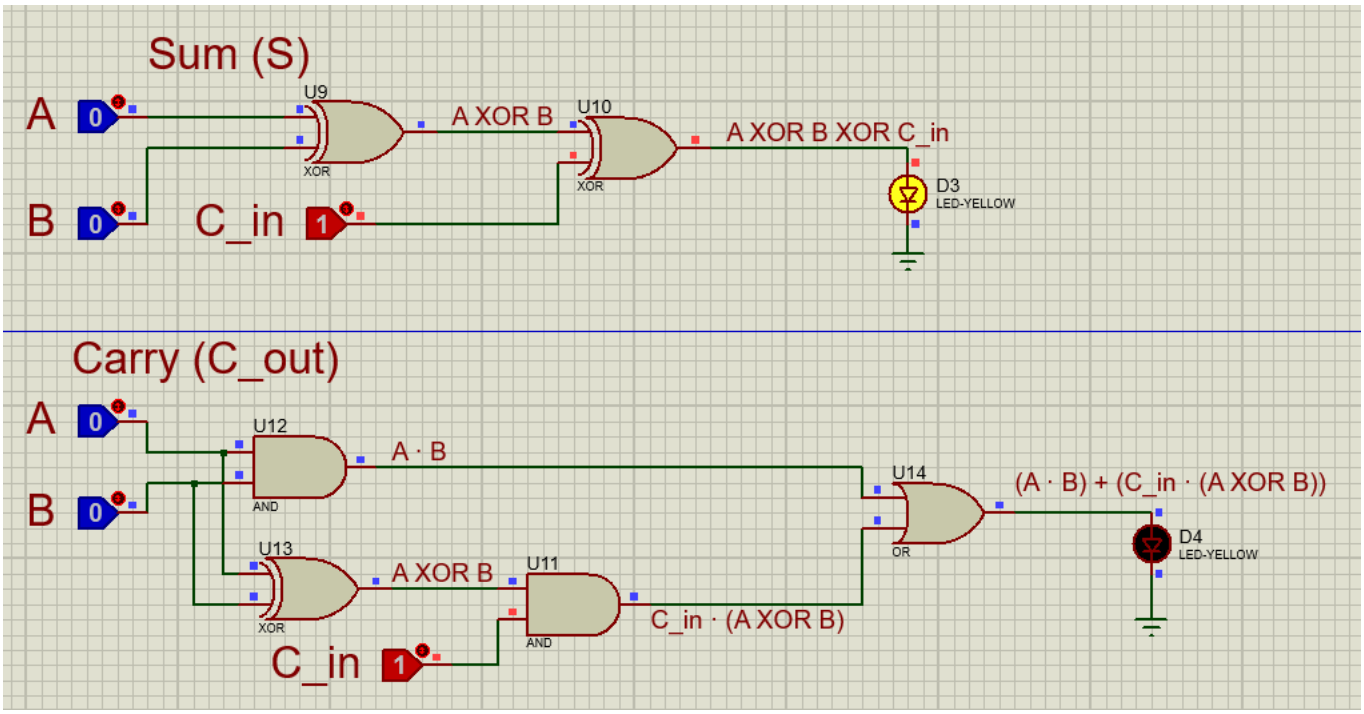
- Use two XOR gates and two AND gates to compute S and C_out.
- Connect A, B, and C_in to the first XOR gate for S.
- Use AND gates to compute (A · B) and (C_in · (A ⊕ B)), then connect to an OR gate for C_out.

iii. Verification

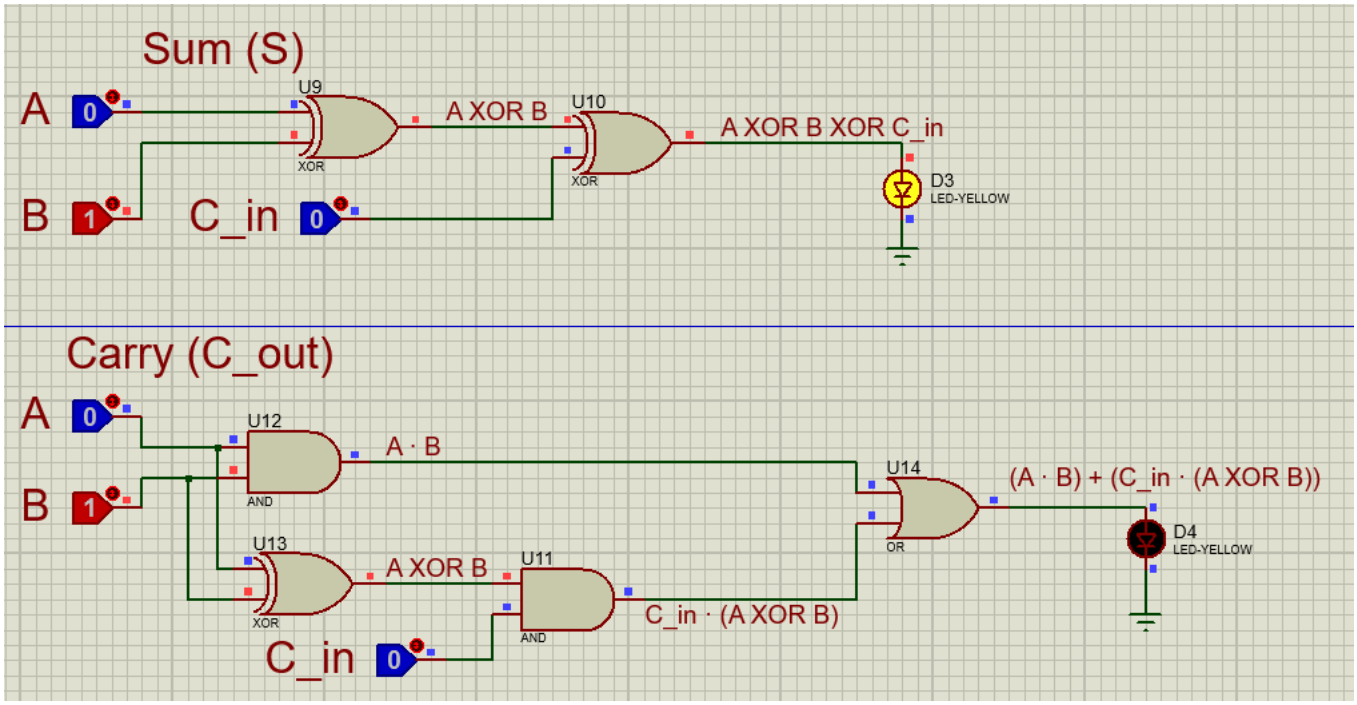
Input: A = 0, B = 0, C_in = 0 → **Output: S = 0, C_out = 0**



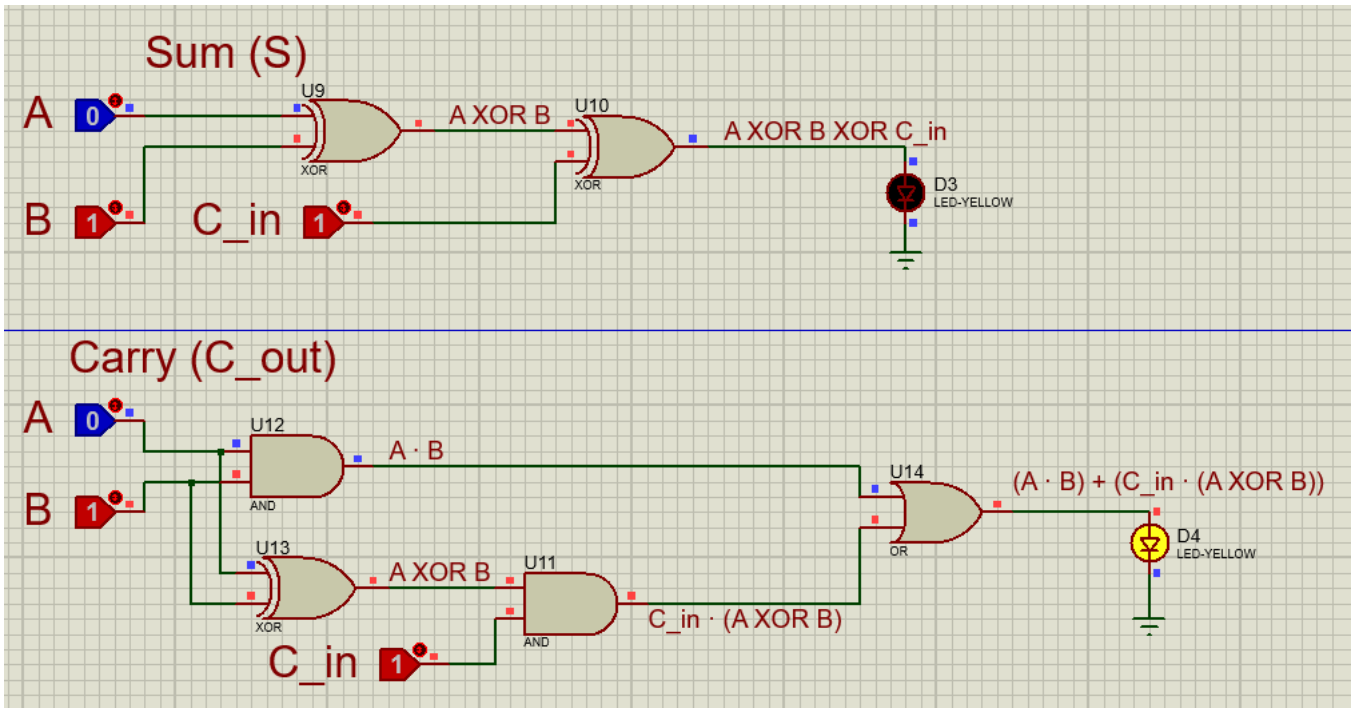
Input: A = 0, B = 0, C_in = 1 → **Output: S = 1, C_out = 0**



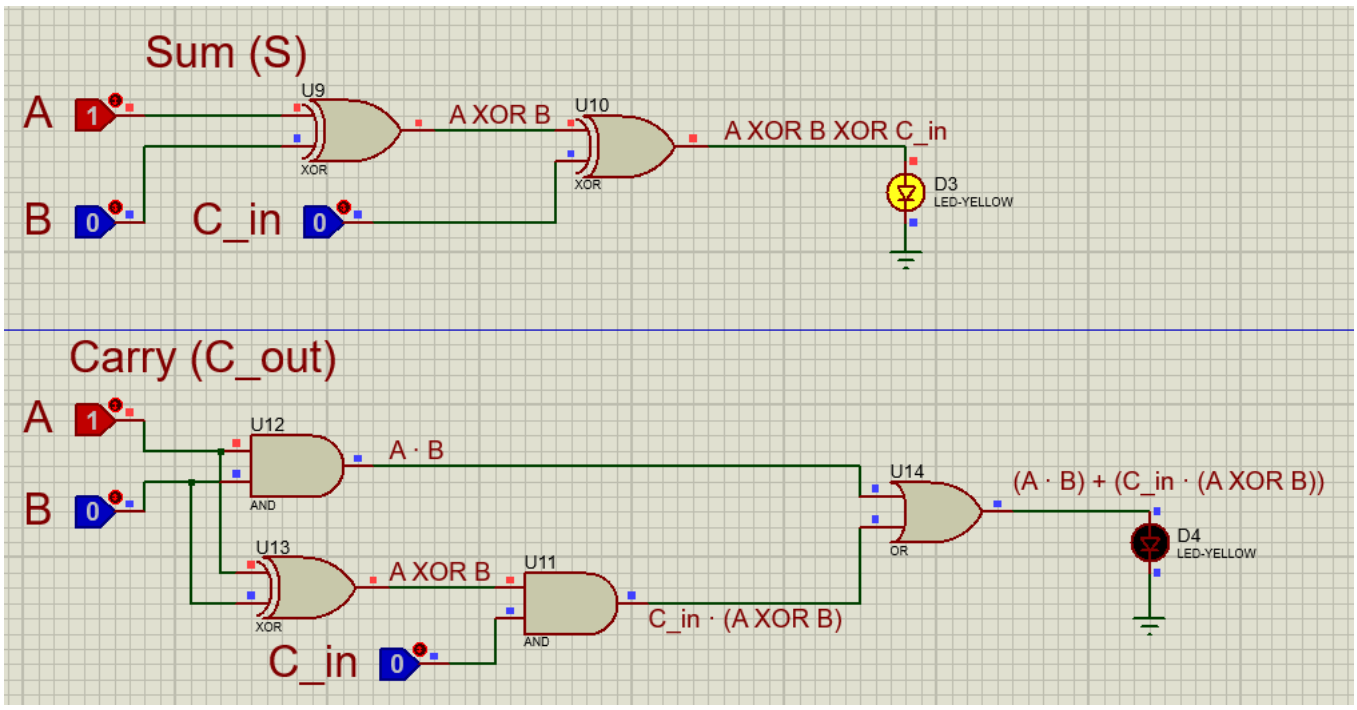
Input: A = 0, B = 1, C_in = 0 → **Output: S = 1, C_out = 0**



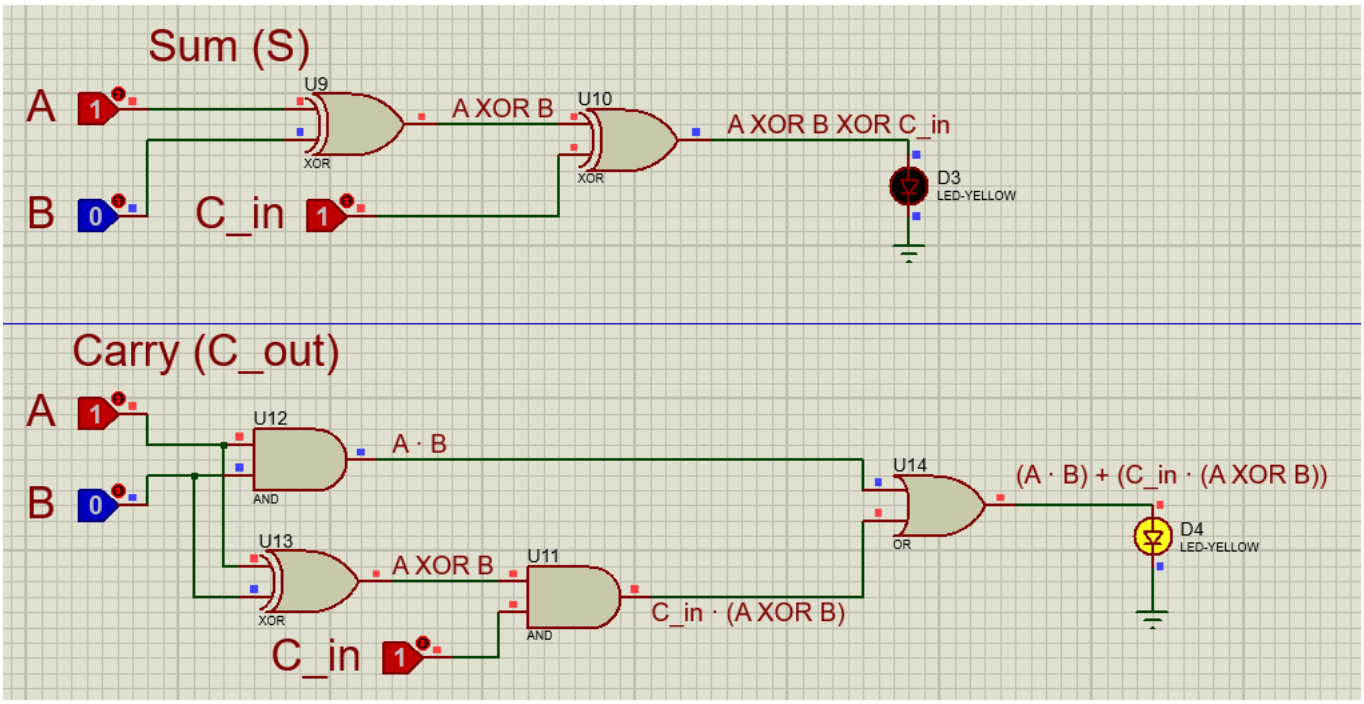
Input: A = 0, B = 1, C_in = 1 → **Output: S = 0, C_out = 1**



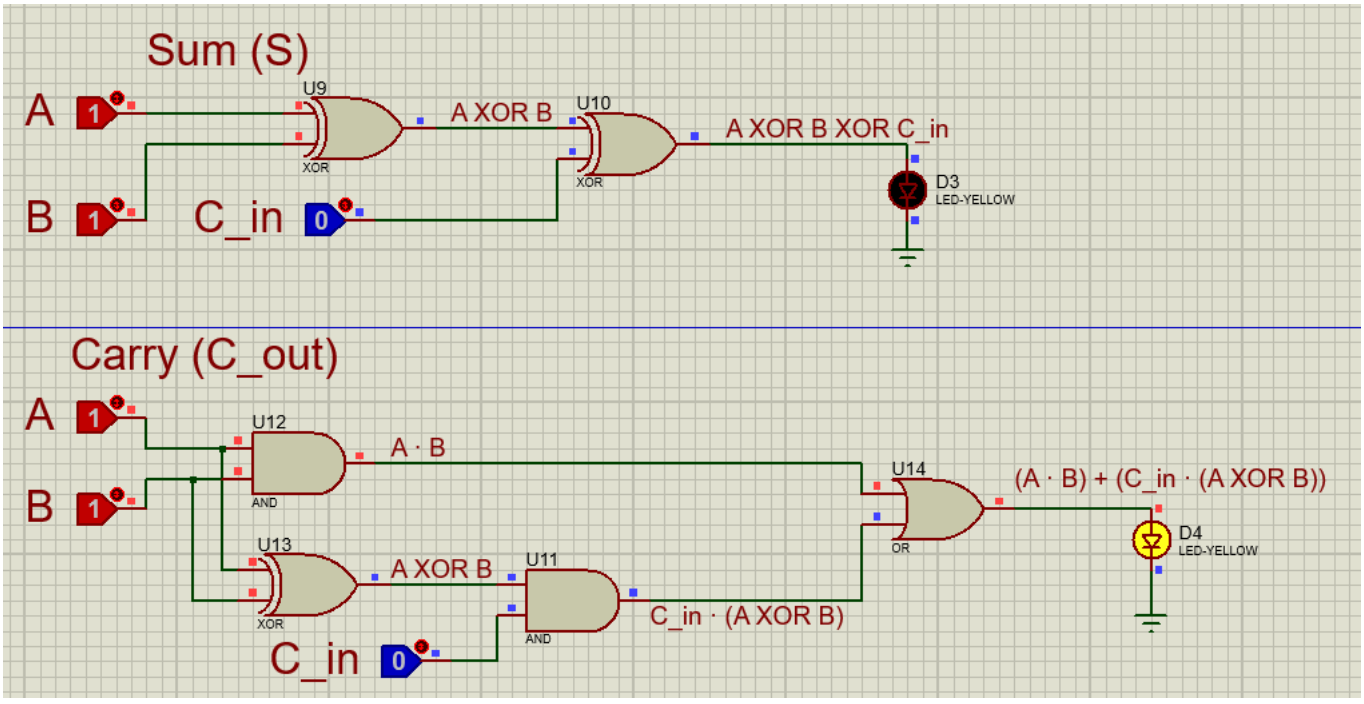
Input: A = 1, B = 0, C_in = 0 → **Output: S = 1, C_out = 0**



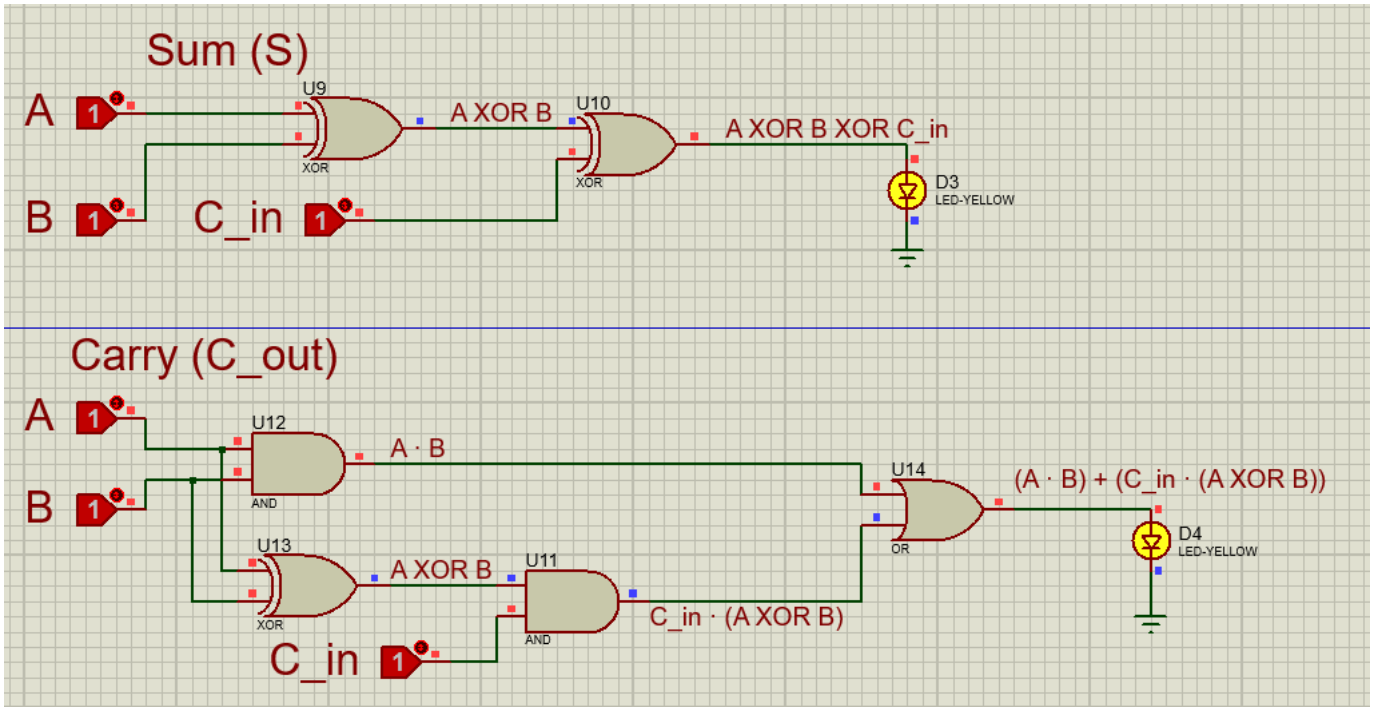
Input: A = 1, B = 0, C_in = 1 → **Output: S = 0, C_out = 1**



Input: A = 1, B = 1, C_in = 0 → **Output: S = 0, C_out = 1**



Input: A = 1, B = 1, C_in = 1 → **Output: S = 1, C_out = 1**



2. Full Subtractor Circuit

i. Truth Table and Logic Expression

A	B	B_in	Difference (D)	B_out
0	0	0	**0**	**0**
0	0	1	**1**	**1**
0	1	0	**1**	**1**
0	1	1	**0**	**1**
1	0	0	**1**	**0**
1	0	1	**0**	**0**
1	1	0	**0**	**0**
1	1	1	**1**	**1**

Logic Expressions:

- Difference (D) = $A \oplus B \oplus B\_in$
- Borrow (B_out) = $(\overline{A} \cdot B) + (B\_in \cdot \overline{A \oplus B})$

ii. Implementation on Proteus:

Components Needed: XOR Gates (7486), AND Gates (7408), OR Gate (7432), NOT Gate (7404).

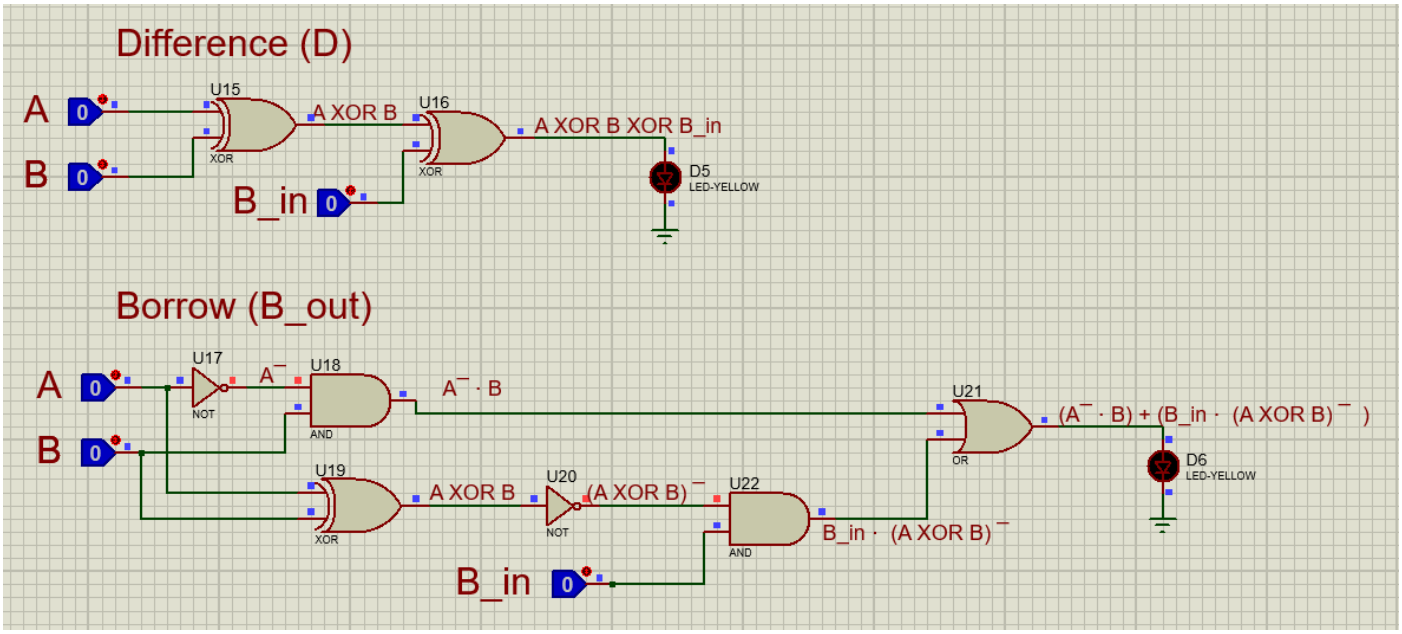
Circuit Design:

- Use two XOR gates for D.

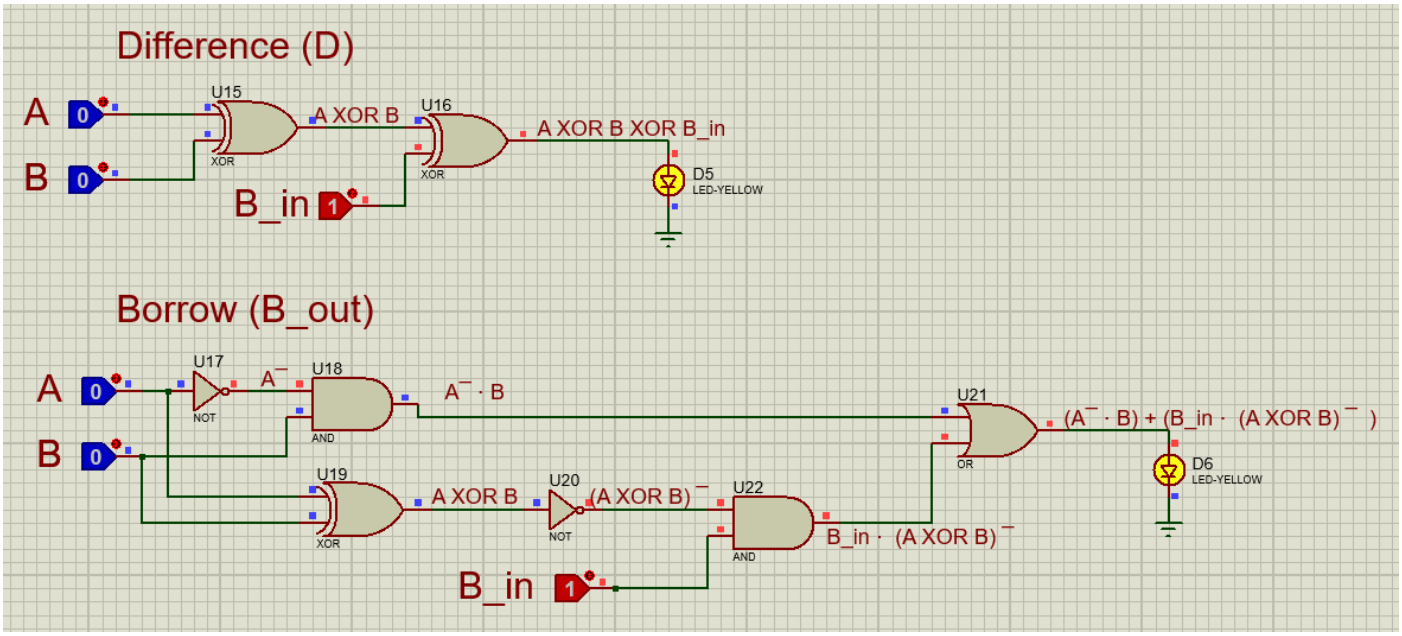
- Use NOT gates for $\overline{A}$, then AND gates for ($\overline{A} \cdot B$) and ($B\_in \cdot \overline{A \oplus B}$), and connect to an OR gate for B_out.

iii. Verification

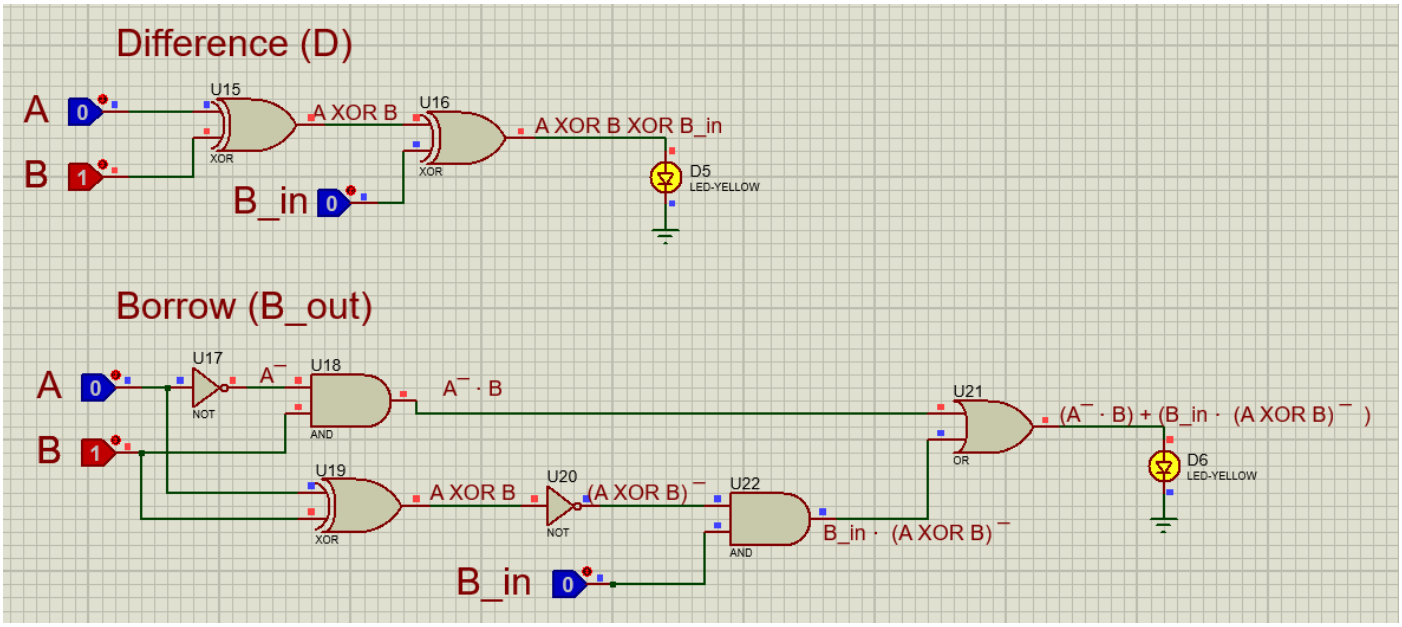
Input: A = 0, B = 0, B_in = 0 → **Output: D = 0, B_out = 0**



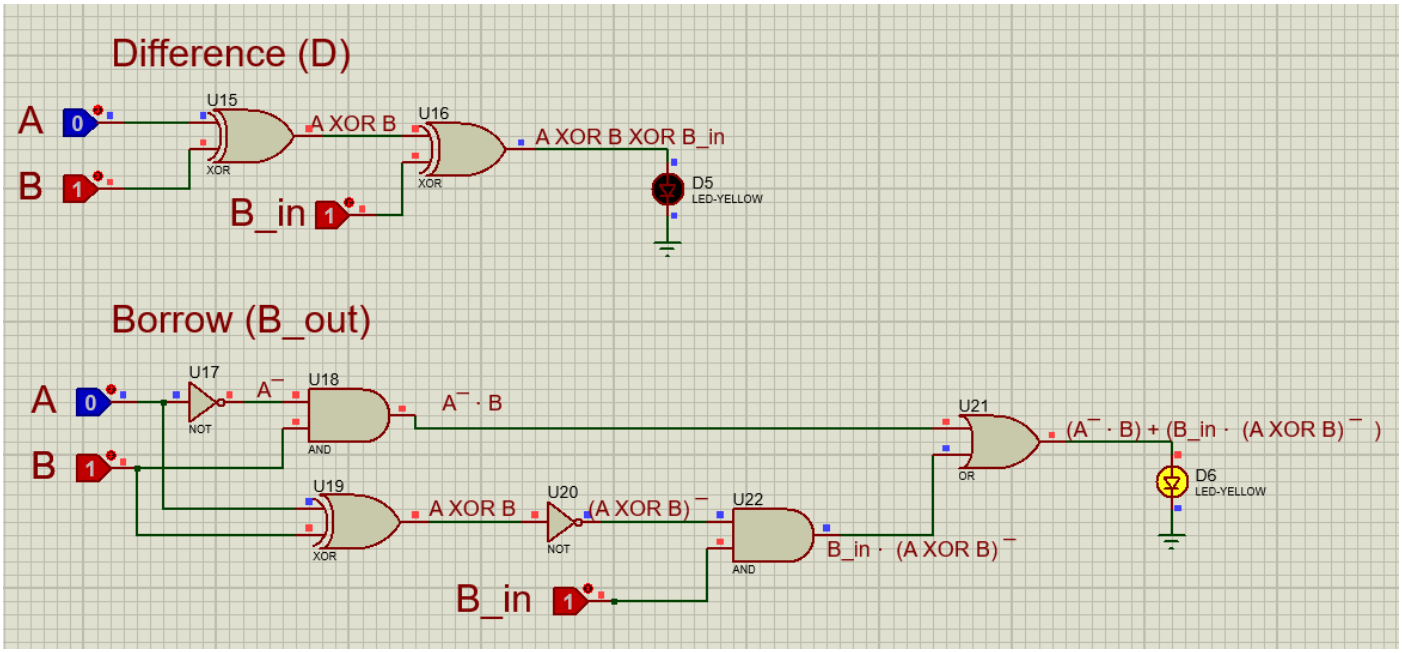
Input: A = 0, B = 0, B_in = 1 → **Output: D = 1, B_out = 1**



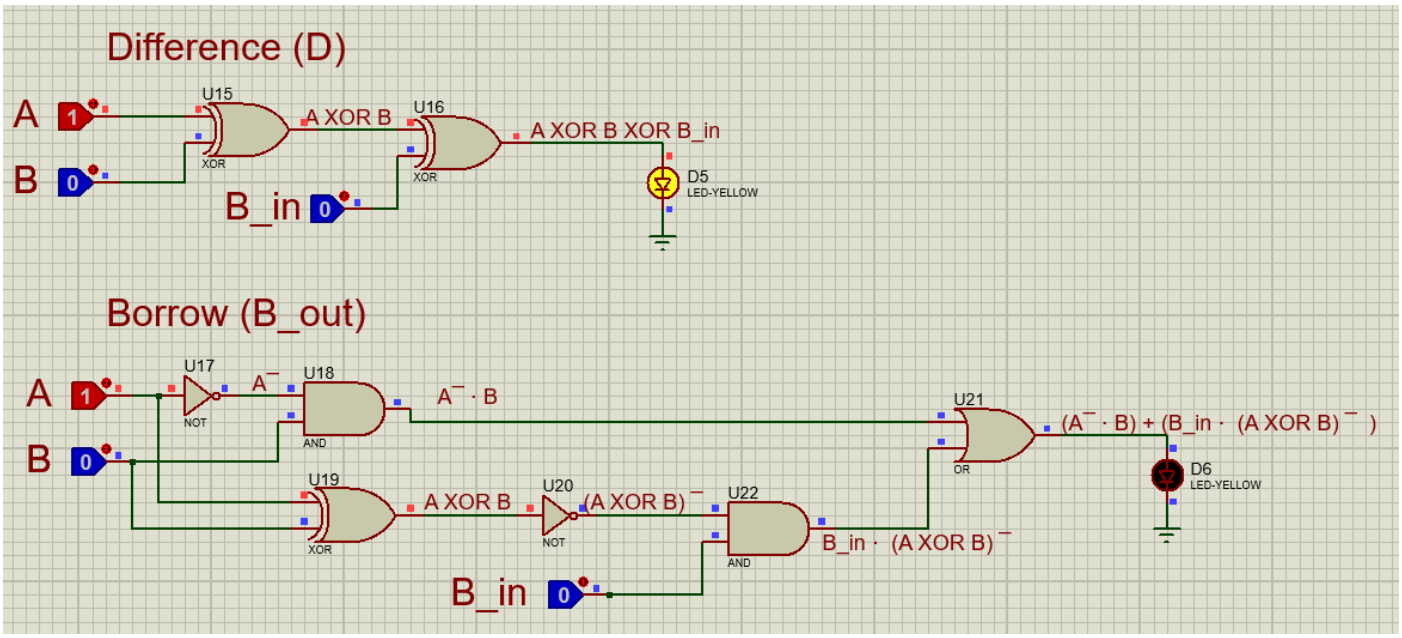
Input: A = 0, B = 1, B_in = 0 → **Output: D = 1, B_out = 1**



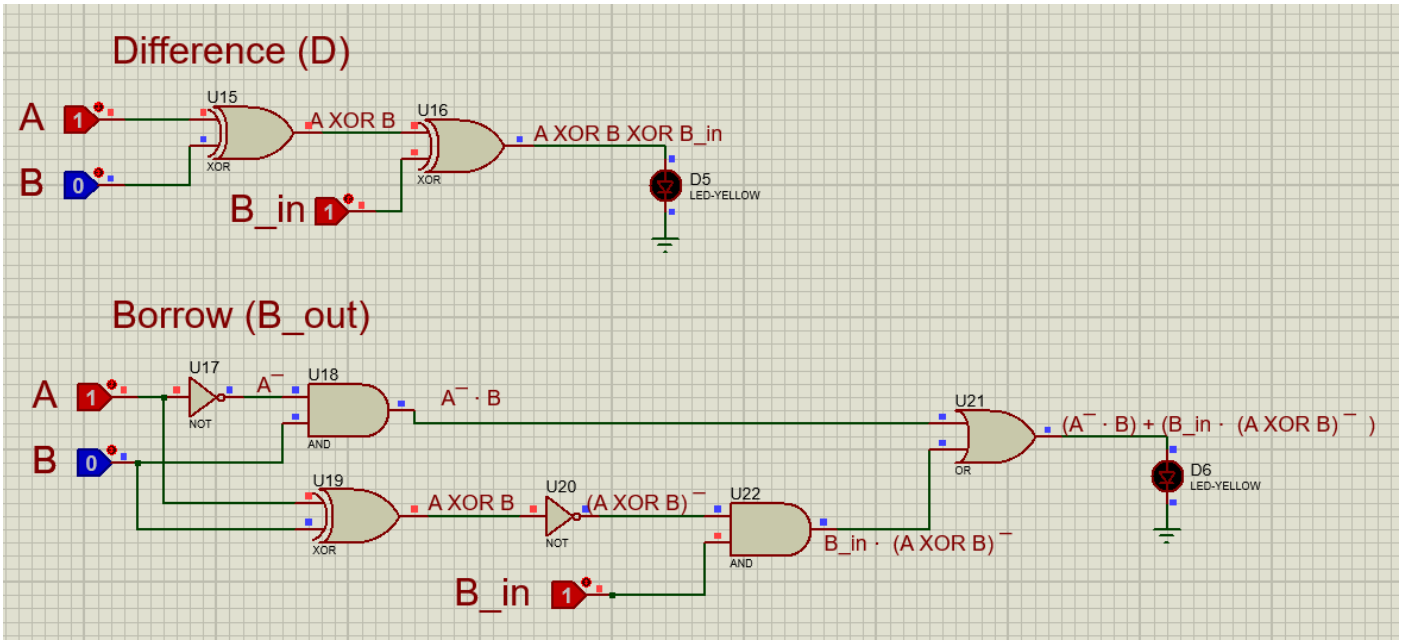
Input: A = 0, B = 1, B_in = 1 → **Output: D = 0, B_out = 1**



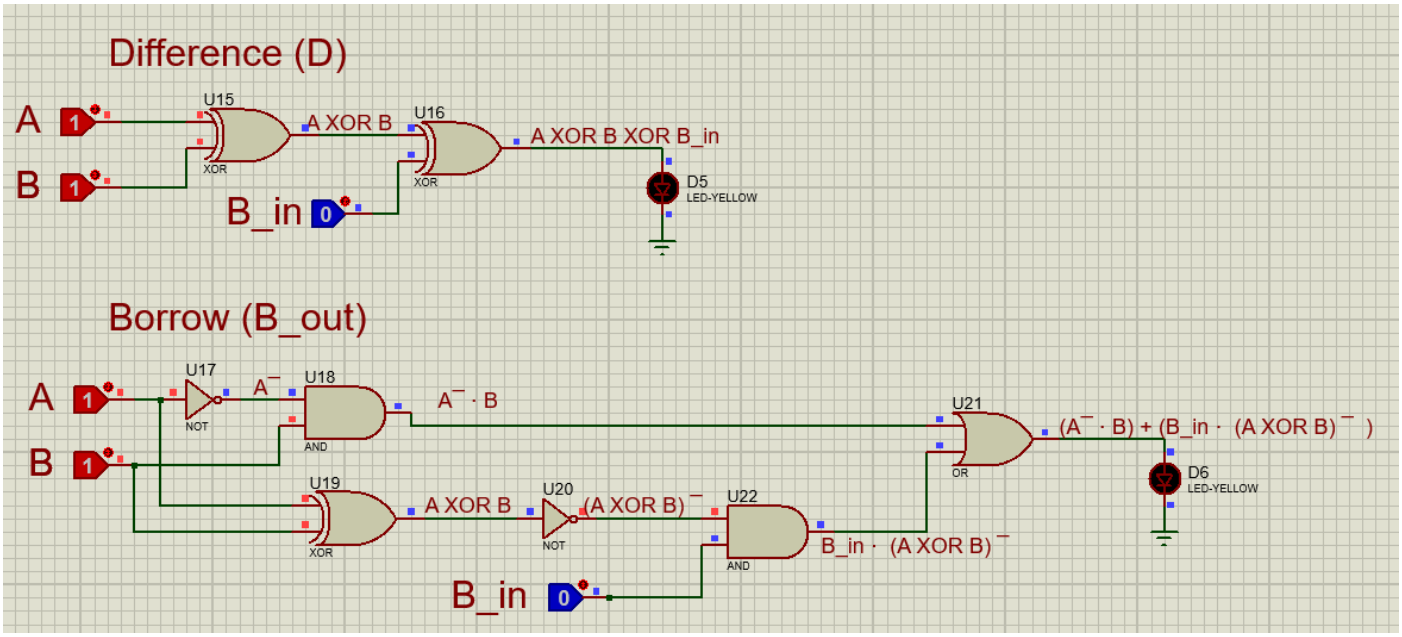
Input: A = 1, B = 0, B_in = 0 → **Output: D = 1, B_out = 0**



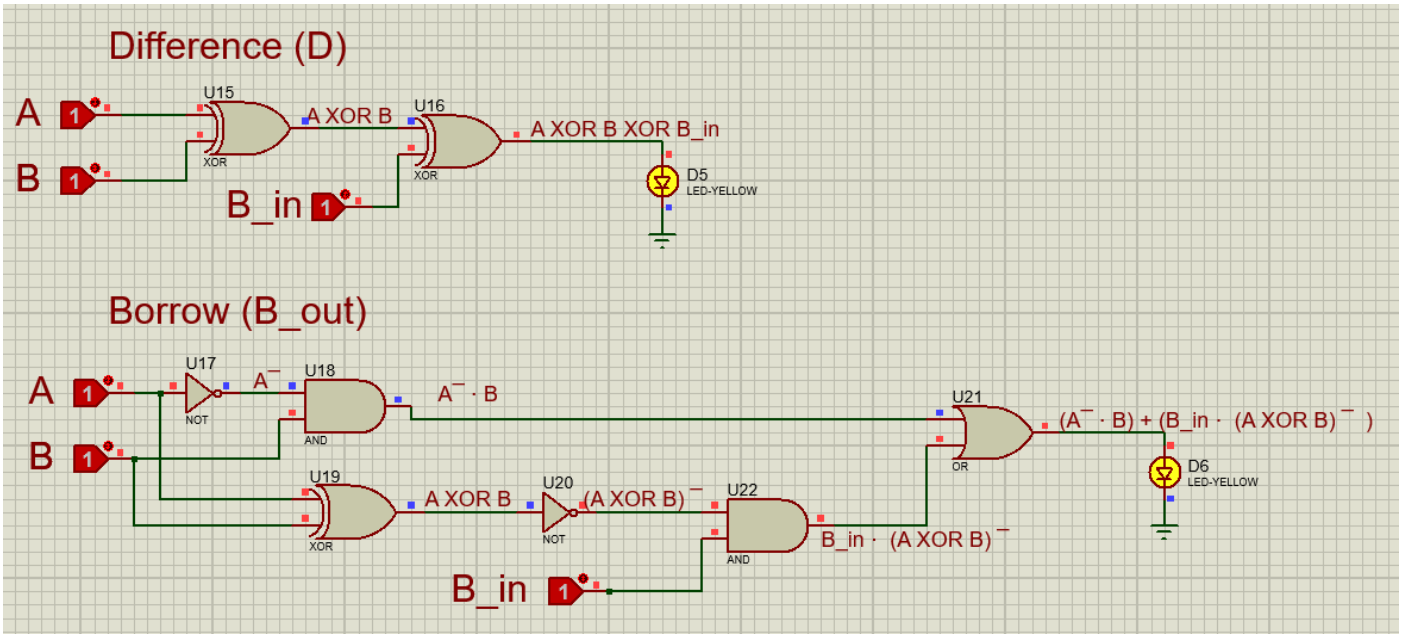
Input: A = 1, B = 0, B_in = 1 → **Output: D = 0, B_out = 0**



Input: A = 1, B = 1, B_in = 0 → **Output: D = 0, B_out = 0**



Input: A = 1, B = 1, B_in = 1 → **Output: D = 1, B_out = 1**



THE END